TECHNICAL DOCUMENTATION

RAG-Based Customer Support Assistant with Human-in-the-Loop

Property	Value
Project Name	RAG-Based Customer Support Assistant
Document Type	Technical Documentation
Version	1.0
Date	April 2026
Author	Abhishek Sambhaji Bhosale
Intern ID	IN226053602
Audience	Engineers, Evaluators, Academic Reviewers

1. Introduction

1.1 What is Retrieval-Augmented Generation (RAG)?

Retrieval-Augmented Generation (RAG) is an AI architecture that bridges the gap between static language model knowledge and dynamic, domain-specific information. It combines two components: a retriever that searches a knowledge base, and a generator (LLM) that produces grounded responses using retrieved context.

Approach	How It Works	Key Weakness	RAG Advantage
Traditional LLM	Query → LLM → Response	Hallucination; static knowledge cutoff	N/A
Fine-tuning	Retrain model on domain data	Expensive; requires retraining on updates	N/A
RAG (This Project)	Query → Retrieve → Context+Query → LLM → Response	Requires vector DB setup	Grounded, updatable, no retraining

1.2 Why RAG for Customer Support?

Challenge	Without RAG	With RAG System
Policy accuracy	LLM may hallucinate incorrect return policies	Responses pulled directly from official PDF
Knowledge updates	Must retrain model or use prompt stuffing	Update PDF; re-embed; no LLM changes needed
Accountability	No source attribution	Answers cite specific document sections
Scale	50 agents for 10K daily queries ($500K/month)	80% automated; reduced agent team needed

1.3 Use Case Scenario

Business Context: An e-commerce company receives 10,000+ daily customer support queries covering returns, product specifications, technical issues, and order status. The existing human-only approach is unsustainable at scale.

Expected Outcomes with RAG System:

- 80% of routine queries answered automatically within 2 seconds
- Human agents only handle the remaining 20% complex/edge-case queries
- Response consistency: all users receive policy-compliant answers
- Operational cost reduction by 60-70% in agent headcount requirements

2. Design Decisions with Rationale

2.1 Chunk Size: 500 Characters with 50-Character Overlap

The choice of chunk size directly impacts both retrieval precision and context completeness. After evaluating multiple configurations:

Chunk Size	Precision	Context Quality	Token Cost	Recommendation
250 chars	High	Poor — cuts mid-sentence	Low	Too fragmented
500 chars (selected)	Good	Good — preserves context	Medium	Optimal balance
1000 chars	Lower	Excellent — rich context	High	Over-retrieves irrelevant text

The 50-character overlap ensures that answers spanning chunk boundaries are not missed during retrieval. This is especially important for multi-part policy statements.

2.2 Embedding Model: text-embedding-3-small

Metric	text-embedding-3-small	text-embedding-3-large	Justification
Dimensions	1536	3072	1536 dims sufficient for support domain
Cost	$0.00002 / 1K tokens	$0.00013 / 1K tokens	6.5x cheaper for same task quality
MTEB Benchmark	~62%	~64%	Marginal 2% improvement not worth 6.5x cost
Latency	~100ms per call	~180ms per call	Faster ingestion pipeline

2.3 Retrieval Parameter: k=4

The number of retrieved chunks (k) directly controls the quality and cost of context provided to the LLM. Empirical testing with 50 sample queries produced the following results:

k Value	Retrieval Accuracy	Avg Latency	Token Cost per Query	Decision
k=2	70%	100ms	Low	Misses critical context
k=4 (selected)	85%	200ms	Medium	Optimal balance
k=8	89%	380ms	High	Marginal gain, more noise
k=10	91%	500ms	Very High	Over-retrieval of irrelevant text

2.4 Confidence Scoring Formula

A deterministic confidence scoring function was implemented to avoid dependency on LLM self-reporting (which is unreliable). The formula is:

```
def calculate_confidence(response: str, retrieved_docs: List[str]) -> float:
    score = 0.5   # base confidence — assume moderate reliability

    # +0.30 if relevant documents were retrieved (primary signal)
    if retrieved_docs and len(retrieved_docs) > 0:
        score += 0.30

    # +0.10 if response is substantive (not a one-liner refusal)
    if len(response) > 50:
        score += 0.10

    # +0.10 if response contains no hedging language
    hedge_words = ['maybe', 'perhaps', 'i think', 'i believe', 'not sure']
    if not any(word in response.lower() for word in hedge_words):
        score += 0.10

    return min(score, 1.0)   # cap at 1.0
```

Threshold of 0.6: Queries scoring below 60% are escalated. This threshold was chosen to balance false positives (unnecessary escalations) against false negatives (incorrect automated responses).

2.5 Prompt Design

The system prompt was designed with four explicit constraints to minimize hallucination and produce appropriately scoped responses:

```
SYSTEM_PROMPT = """You are a customer support assistant for an e-commerce company.

CONTEXT:
{context}

USER QUESTION: {query}

INSTRUCTIONS:
1. Answer ONLY using the context provided above
2. If the context does not contain the answer, respond:
   'I don't have that information in our knowledge base.'
3. Be concise, helpful, and professional
4. Do not invent policies, prices, or procedures

ANSWER:"""
```

Prompt Element	Purpose	Without It
Role assignment	Defines persona and domain scope	LLM may answer out-of-scope questions freely
Explicit context boundary	Prevents hallucination beyond PDF content	LLM invents policies not in the document
Fallback instruction	Graceful handling when answer is not found	LLM makes up an answer rather than admitting
Anti-invention instruction	Reinforces factual grounding	LLM adds false details with confidence

3. LangGraph Workflow — Deep Dive

3.1 Why LangGraph Over Simple Function Chain?

Capability	Simple Function Chain	LangGraph StateGraph
Conditional routing	Complex if/else nesting	Clean conditional_edges declaration
State management	Pass dict manually through all functions	Typed state automatically flows between nodes
Conversation memory	Manual implementation required	Built-in MemorySaver with thread_id
Debugging	Print statements only	Visual graph + execution trace
Extensibility	Hard to add new branches	Add node + edge declaration

3.2 State Progression Through Workflow

This shows how the QueryState evolves as it passes through each node in the graph:

```
# Initial state (before any node execution)
{ 'query': 'How do I return a product?', 'retrieved_docs': [], 'context': '',
  'response': '', 'confidence_score': 0.0, 'needs_escalation': False, 'intent':
'' }

# After retrieve_node
{ ..., 'retrieved_docs': ['Return policy: 30 days...', 'Refund timeline: 5-7
days...'],
       'context': 'Return policy: 30 days...\n\nRefund timeline: 5-7 days...' }

# After generate_node (high confidence path)
{ ..., 'response': 'You can return products within 30 days of purchase...',
       'confidence_score': 0.90 }

# Router decision: 0.90 >= 0.60 → respond → END (no escalation)
```

4. Human-in-the-Loop (HITL) Implementation

4.1 Motivation for HITL

No RAG system achieves 100% accuracy. HITL is essential for production deployment to handle: edge cases not covered by the PDF, complex multi-part queries requiring judgment, emotionally sensitive situations requiring human empathy, and queries with potential legal or compliance implications.

4.2 Escalation Ticket Lifecycle

Lifecycle Stage	Actor	Action	System State
Query received	User	Submits ambiguous or complex query	Query in processing
Low confidence detected	LangGraph router	Routes to escalate node	confidence < 0.60
Ticket created	HITLManager	Writes ticket to escalations.json	status = 'pending'
User notified	Streamlit UI	Shows ticket ID and wait message	User waiting

Lifecycle Stage	Actor	Action	System State
Agent reviews	Human Agent	Opens dashboard, reads query + LLM attempt	Agent reviewing
Resolution submitted	Human Agent	Submits response via text area + button	status = 'resolved'
Response visible	Streamlit UI	Shows resolved response in dashboard	Ticket closed

4.3 HITL Benefits vs. Limitations

Benefit	Description
Quality Safety Net	Human agents catch and correct AI errors before they reach frustrated customers
Edge Case Learning	Resolved tickets become training data for future system improvements
Compliance	Sensitive queries (legal threats, medical, fraud) always get human review
User Trust	Users feel confident that complex issues will receive human attention
Limitation	**Mitigation Strategy**
Response delay (minutes vs. seconds)	Immediately acknowledge with ticket ID and expected response time
Agent availability constraints	Implement on-call schedule; push notifications to mobile
Queue backlog during peak hours	Priority scoring based on sentiment analysis of query content
Agent consistency	Provide standardized response templates for common escalation types

5. Challenges and Trade-offs

5.1 Cost vs. Model Quality

Model	Cost/Query	Avg Latency	Quality	Final Decisi
GPT-3.5-turbo (selected)	$0.002	~1.5s	Good for structured support	Selected — 15x chea
GPT-4 Turbo	$0.01	~2.5s	Excellent reasoning	Overkill for FAQ-style
Local LLaMA-3 (8B)	$0 (compute)	~4s (CPU)	Moderate	Unacceptable latenc

5.2 Local vs. Cloud Vector Store

Factor	ChromaDB (Local)	Pinecone (Cloud)
Setup time	5 minutes (pip install)	30+ minutes (account, API, indexes)
Cost	Free	$0.10+/hour for production tier
Scalability	Limited to disk + RAM	Auto-scales to billions of vectors
Latency	~200ms (local disk I/O)	~50-100ms (optimized cloud infra)
Decision	Best for demo/development	Recommended for production deployment

5.3 Key Technical Limitations

- PDF Quality Dependency: System performance degrades significantly with scanned PDFs (requires OCR preprocessing)
- Single-Language Support: Embedding model optimized for English; multilingual support requires different model
- Static Knowledge Base: Adding new information requires re-running the full ingestion pipeline
- No Conversation Memory Across Sessions: MemorySaver is in-memory only; restarts clear conversation history
- Confidence Scoring is Heuristic: The scoring formula is rule-based, not ML-trained; may not generalize to all domains

6. Testing Strategy

6.1 Test Coverage Plan

Category	Number of Tests	Tooling	Pass Criteria
Unit Tests	15 tests	pytest	100% pass rate
Integration Tests	8 tests	pytest + asyncio	100% pass rate
End-to-End Scenarios	5 manual flows	Manual + Streamlit UI	All scenarios verified
Performance Tests	3 benchmarks	time.perf_counter ()	Latency targets met

6.2 Manual Test Scenarios

Scenario	Steps	Expected Outcome	Pass/Fail Indicator
Normal query flow	Ask 'What is your return policy?'	Direct answer with confidence > 60%	No escalation warning shown
Escalation flow	Ask 'I want to file a complaint'	Escalation message + ticket ID	Ticket appears in agent dashboard
Agent resolution	Open dashboard, submit response	Ticket status changes to resolved	Status shows 'resolved'
Empty knowledge base	Delete chroma_db/, run query	Error handled gracefully	User sees helpful error message
Long PDF ingestion	Load 100+ page PDF	Successful chunking within 30s	Vector count confirms all chunks stored

7. Future Enhancements

7.1 Short-Term (1-2 Months)

Enhancement	Description	Estimated Effort
Multi-PDF support	Upload and query across multiple knowledge base documents simultaneously	Medium (3-5 days)
Persistent chat history	Store conversation history in SQLite for cross-session continuity	Low (1-2 days)
User feedback loop	Thumbs up/down on responses to improve confidence scoring model	Low (1-2 days)
Response caching	Cache embeddings for frequently asked queries to reduce API calls	Medium (2-3 days)

7.2 Long-Term (6-12 Months)

- Real-time agent chat: WebSocket integration for live handoff from AI to human agent
- Voice interface: Speech-to-text input + text-to-speech output for phone support
- Analytics dashboard: Query volume trends, escalation rates, avg confidence, resolution times
- Active learning: Automatically fine-tune embedding model using resolved escalation tickets
- Multi-language support: Integrate multilingual embedding models (e.g., multilingual-e5-large)
- Production deployment: Docker containerization + Kubernetes orchestration on AWS/GCP

8. Conclusion

The RAG-Based Customer Support Assistant successfully demonstrates the practical application of modern AI techniques in a real-world business context. The project integrates document intelligence, stateful workflow orchestration, and human oversight into a cohesive, production-oriented system.

8.1 Key Technical Achievements

- Built a complete end-to-end RAG pipeline from PDF ingestion to user-facing chat interface
- Implemented stateful multi-node workflow using LangGraph with conditional escalation routing
- Designed a deterministic confidence scoring system that routes uncertain queries to human agents
- Created a functional HITL dashboard enabling agent review and resolution of escalated tickets
- Achieved 85% retrieval accuracy with 2-second average response time in testing

8.2 Key Performance Metrics

Metric	Value	Notes
Retrieval Accuracy (k=4)	85%	Tested on 50 sample queries
Average Response Latency	~2 seconds	End-to-end including LLM call
Confidence Threshold	60%	Queries below escalated to human
Automation Potential	~80%	Routine queries handled without human intervention
Document Ingestion (50 pages)	~15 seconds	One-time setup operation

8.3 Appendix: Troubleshooting Guide

Error	Likely Cause	Solution
ModuleNotFoundError	Dependencies not installed	Run: pip install -r requirements.txt
OpenAI API error 401	Invalid API key	Check OPENAI_API_KEY in .env file
ChromaDB persistence error	Corrupted vector store	Delete chroma_db/ folder; re-run ingestion
Port 8501 already in use	Another Streamlit instance running	Run: streamlit run app.py --server.port 8502
PDF text not extracted	Scanned/image PDF	Convert PDF to text-based using OCR preprocessing
Low confidence on all queries	Empty or irrelevant knowledge base PDF	Verify PDF content matches query domain